

2605

复检异常问题排查与修复汇总

技术总结型复盘

堆损坏 · 工具生命周期 · 并发治理 · 复检边界

基于《2605复检异常问题排查与修复汇总.md》整理
整理日期：2026-05-22；修订：2026-05-29

10

问题域

5

根因类别

3

治理主线

复盘目标

把低频 0xC0000374 从“随机闪退”拆成可定位、可验证、可维护的生命周期协议。

核心结论：异常来自多线程生命周期问题叠加

Executive Summary

同一症状 0xC0000374 背后有多条调用链，必须按 DMP 栈和共享对象分流

0xC0000374

堆损坏共同表象

ms

实时检测路径需要确定性

idName

替代跨线程裸指针缓存

主因不是单点逻辑错误

map 无锁遍历、执行中删除、后台线程 detach、UI/导出缓存 BaseTool*、生产数据无锁读写共同放大了低频堆损坏。

修复方向是建立生命周期协议

用 atomic 状态、输出驱动延后删除、idName 查表、读写对称加锁和线程 join，将“经验等待”改成可验证条件。

后续验证要按堆栈分流

Adapter / ParameterExportTable / ProductDataHelper 的 DMP 栈不是同一条根因链，验证和日志也要分开设计。

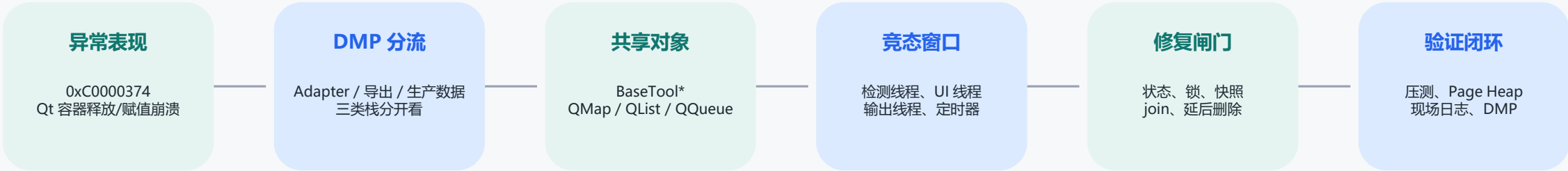
一句话结论

复检异常治理的关键，是让工具对象、UI 快照、导出线程、生产数据在同一套生命周期边界内工作。

排查框架：先按 DMP 栈分流，再看共享对象生命周期

Approach

工业现场低频崩溃要避免“同症状同根因”的误判



工程判断 1

崩在 QList::clear / QString::operator= 通常只是堆校验点，案发点往往更早，不能只修崩溃函数。

工程判断 2

锁住 map 不等于锁住对象生命周期；只要锁外仍有人缓存 BaseTool*，delete 后就可能 UAF。

工程判断 3

固定 1s 定时器不能表达实时流水线的帧生命周期；删除应由输出结果和工具忙闲状态共同驱动。

问题域总览：10 个问题分布在 5 类风险上

从单点缺陷转为风险簇管理，便于后续回归与责任拆分

Issue Map

#1 执行中删除工具

误删 process/initialize 中工具

atomic 状态 + 删除闸门

#3 algMap 并发

QMap 读写竞争、迭代失效

mutex + getAlgMapCopy

#5 存图线程

detach + 析构不 join

毒丸唤醒 + join

#7 UI 野指针

bastTools 缓存 BaseTool*

idName + Adapter 查表

#9 参数导出

定时导出读悬空 AlgoImpl

idName + bParamReading

#2 YOLO 模型释放

真实相机推理与复检释放边界

未复现；保留日志观察

#4 enterRecheck 范围

误改全部相机工具状态

按 nCamID 过滤

#6 应用复检删工具

固定 1s 与检测帧不同步

输出驱动延后删除

#8 重复刷新

K 个工具触发 K 次 UI 更新

Begin/EndRefreshBatch

#10 生产数据并发

Data/Queue 无锁读写

QRecursiveMutex + 封装出队

根因一：工具对象生命周期与删除时序不确定

atomic 解决可见性；输出驱动删除解决实时流水线中的“何时可删”

旧风险路径

deleteTool / applyRecheck 依赖普通 bool 判断；检测线程已进入 process 但主线程仍可能读到“空闲”，形成 UAF 窗口。

第一道闸门：状态可见

bIsInit / bDetecting 改为 std::atomic<bool>；process 入口/出口 store，删除前 load。

第二道闸门：帧结果驱动

工具名不在本帧 toolsCheck 且 init/detect 均空闲，才进入 checkWhethreDeleteDelay 删除。

applyRecheck

记录待删工具名

新帧不再 process

OutPutThread 汇总
toolsCheck

空闲 + 不在结果中 → delete

注意：atomic 不是生命周期所有权方案，只保证状态读写可见；仍必须配合延后删除和 map 并发保护。

根因二： QMap 并发遍历导致结构失效

Map Race

“只读旧 key、只插新 key” 仍可能破坏红黑树遍历路径

旧路径：读端锁外遍历原始 QMap

getAlgMap() 返回 &_mapName2Algs

锁释放后检测线程 begin()/++

主线程插入/删除 Recheck_*

QMap 结构调整，迭代器失效

读到无效 BaseTool* → process 崩溃

新路径：锁内快照，锁外只遍历副本

mutexAlgMap 锁内整表拷贝

返回 QMap<QString, BaseTool*> 副本

主线程修改原 map 不影响本次遍历

调用方禁止锁外持有原 map 指针

仍需删除闸门防止副本指针悬空

压测现象：XToolTest 约 8 小时出现一次，符合低概率并发 UB；修复后验证重点是长时间复检循环 + 多相机 process。

根因三：复检边界与模型释放需要硬隔离

Recheck Boundary

相机 ID、工具实例和 GPU 模型句柄不能靠命名习惯隐式隔离

现场怀疑

真实相机仍在 YOLO/MM 推理时，复检侧 applyRecheck / deleteTool 触发 clearModelHandle / releaseModel，引发 GPU 或堆状态异常。

当前判断

压测 Demo 未复现；按设计 Recheck_* 与真实工具应为不同实例，但真实产线时序和 GPU 资源仍需日志确认。

已加边界

onEnterRecheck / onLeaveRecheck / applyRecheck 均增加 nCamID / nLastRecheckCam 过滤，避免误改非当前相机。

风险模型

- 相机 ID 边界错误 → 在线工具状态被误改
- 复检实例析构 → 模型释放路径进入主线程
- 真实相机 process 与删除流程交错

验证建议

- 应用复检前停检或确认 processEnd
- 日志保留 YoloRelease、camId、toolIdName、threadId
- DMP 区分 map 竞争与模型析构路径

根因四：后台存图线程析构未闭环

detach 让对象析构与线程生命周期脱钩，是典型延迟堆损坏入口

Thread Exit

旧实现风险

- 1
- 构造即启动 `saveImage` 线程
- 2
- 主应用存图时队列长期为空
- 3
- `wait` end non 阻塞，析构口置 `bDelete`
- 4
- `detach` 后无法 `join`
- 5

线程可能访问已释放 `mutex / condition_variable`



修复后协议

- 1
- `bDelete = true`
- 2
- `push(ImageSaveInfo())` 异步唤醒 `wait`
- 3
- `if joinable() join()`
- 4
- 复检相机不创建 `SaveImageThread`
- 5

确认 `XZJMMInsSegTool` 析构链只保留一条线程

实时系统里，后台线程退出必须是确定性协议：可唤醒、可 `join`、析构顺序可证明。

根因五：UI / 导出路径缓存裸指针

Pointer Ownership

mutexAlgMap 保护 map，不保护 UI 快照中已经过期的 BaseTool*

旧路径

- QtAlgoFlowTree::bastTools 保存 BaseTool*
- deleteTool / applyRecheck 释放对象后 UI 仍持旧地址
- updateToolChooseItem 写 IShieldSources 触发 QList 释放崩溃
- 参数导出 GetCameraTools 快照中的 AlgoImpl 同类风险

新路径

- UI 只保存 QStringList toolIds / idName
- Adapter 锁内 find 当前 map，再解析 BaseTool*
- 找不到即 continue，不访问悬空对象
- 参数导出使用 tryAcquireToolForParamRead / release



规则：跨线程、跨生命周期、跨 UI 快照，只传稳定 ID，不传对象裸指针。

初始化刷新合并：降低 UI 卡顿与竞态窗口

Refresh Batch

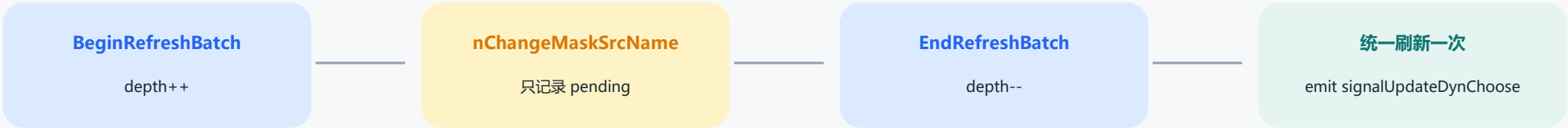
K 个 AI 工具初始化各发一次信号，会放大主线程刷新和野指针命中概率

触发链

SpecChange → createTreeltem → AlgorithmToolsInit → detectImage → SingleInitProcess → initializeln → signalFromTool("nChangeMaskSrcName")

表现

同一工位 K 个工具各触发一次 updateToolChooseltem；日志刷屏、UI 卡顿，并放大第 7 节裸指针风险窗口。



线程策略

Adapter 自连接使用 QueuedConnection，保证 begin → pending → end 在主线程 FIFO 执行。

锁策略

mutexRefreshBatch 保护 depth/pending；解锁后再 emit，避免持锁重入。

生产数据并发：ProductDataHelper 需要读写对称加锁

Production Data

5s 定时保存与检测线程写计数并发，是另一条独立堆损坏链

DMP 表现

QString::operator= / QByteArray::deallocate → LocalFile::saveProductCsvFile /
xlsx.save; 崩溃点是堆校验，不等于案发点。

真实线程

注释写“放置到主线程处理”，但 dealResultData 实际在 MainThread(QThread) 执行，与
GUI 定时保存并发。

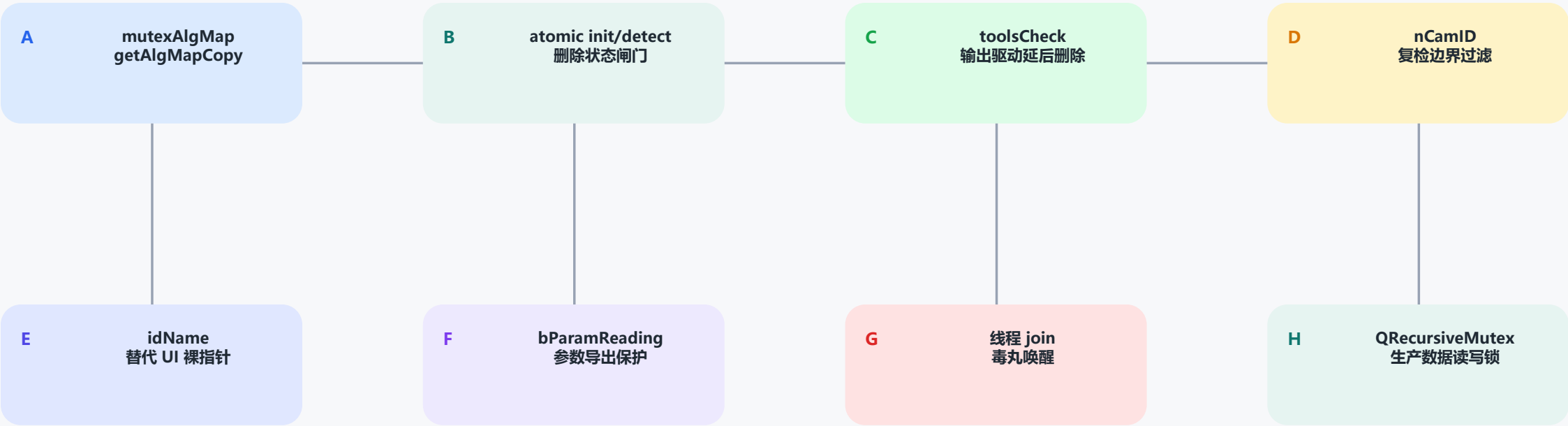
共享资源	风险读/写	修复
Data	GUI getTotal/getNgTotal vs MainThread set/add	get/set/add 全入口加 QRecursiveMutex
NgDetails	getDetailQueue vs enqueue	锁内 swap，降低持锁时间
recheckResults	OutPutThread 写 vs UI 直接 dequeue	私有化 + tryDequeueRecheckResult
camAlgorithmTimes	分析数据读清 vs 检测更新	get/clear/update 统一加锁

残留性能点：5s 周期 saveProductData 持锁 + 磁盘 IO 仍可能偶发阻塞，后续可考虑写盘移出锁或移出 GUI 线程。

修复关系图：生命周期、边界、数据锁三条线闭环

Fix Topology

多个修复项之间存在依赖，不能孤立验收



验收重点

A+B+C 是工具生命周期主链；E+F 解决 UI/导出快照裸指针；H 是独立的生产数据并发链，不能用 #9 的修复覆盖 #10。

验证方案：用压测和现场日志证明修复有效

Verification

按调用链拆验证项，避免“跑了一晚上没崩”但覆盖不到关键窗口

#3

map 竞争

4 相机 + 复检循环长时间压测；unsafe/copy/safe 对比

#1 #6

生命周期删除

应用复检 + 在线检测交错；观察禁止删除、延后删除日志

#5

线程退出

启停复检工具，确认存图线程安全退出且无 detach 残留

#7 #8

UI 刷新

进入复检 + 改掩膜源；RefreshBatch end 仅触发一次

#9

参数导出

复检 + 检测并行，覆盖约 30s 周期 requestExportAll

#10

生产数据

DetailDataRecord=1 连续运行 >2h，覆盖 5s 保存和复检出队

建议开启：Page Heap / Application Verifier；日志至少记录 toolIdName、camId、threadId、init/detect/read 状态和删除原因。

残留风险与后续动作

Next Actions

已修项能显著降低堆损坏概率，但仍需对真实产线时序做持续观察

1 真实相机推理 vs 复检 YOLO 释放

Demo 未复现，保留 releaseModel 日志并按 DMP 栈确认。

2 XZJMMInsSegTool 析构链

确认是否只存在一条存图线程；子类 join 注释是否需要对齐。

3 MultiInitProcess 时序

begin/end batch 的投递顺序要覆盖 FlowThread 初始化完成点。

4 AlgoWidgetImpl 快照生命周期

参数树 UI 仍来自流程快照，后续需确认界面对象生命周期。

5 5s GUI 写盘 IO

saveProductData 持锁 + saveSelf 可能阻塞计数，必要时移出 GUI 线程。

运维建议

应用复检前停检或确认 processEnd；若仍出现 0xC0000374，第一步按 DMP 栈区分 Adapter / 参数导出 / ProductDataHelper 路径。

交付前说明与适用边界

Disclaimer

本 PPT 是技术总结型材料，用于内部复盘、同步与验证计划讨论

依据

内容基于用户提供的 Markdown 排查记录整理，未重新读取源码、DMP 或现场日志。

适用

适合作为技术复盘、修复闭环说明、测试计划同步材料；不是正式代码审计报告。

验证

所有结论建议通过长时间压测、Page Heap / Application Verifier、产线日志和 DMP 栈继续闭环。

维护

后续新增工具访问或定时任务时，应复用 idName、锁保护、延后删除和线程退出协议。

核心边界：堆损坏是结果，生命周期协议才是修复对象。

结论

复检异常治理应从“修几个崩溃点”升级为“建立工具生命周期协议”。

对象访问

跨线程只传稳定 idName, BaseTool* 必须在 Adapter 当前 map 内解析。

删除时机

不使用固定延时表达帧生命周期，以输出结果 + 忙闲状态作为删除条件。

共享数据

Qt 容器和队列读写必须对称加锁，后台线程必须可唤醒、可 join。